Software Engineering

UML Aktivitätsdiagramm

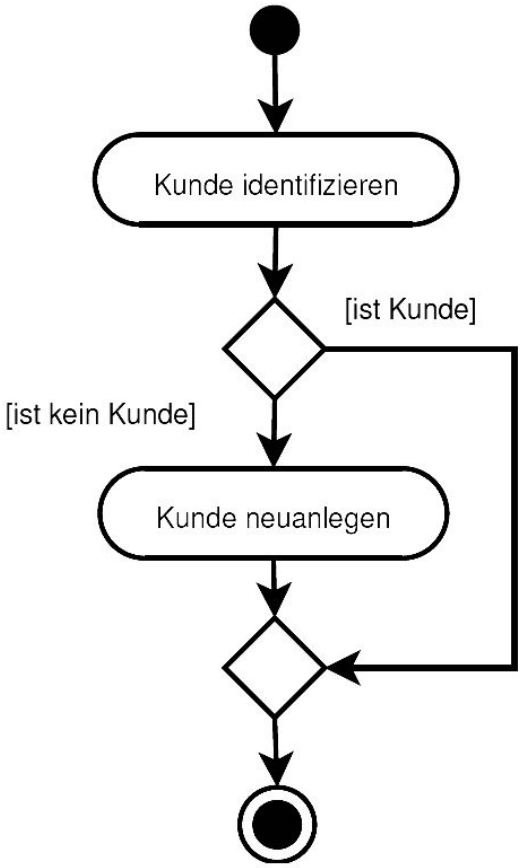
Prof. Dr.-Ing. Sven Graupner



UML Aktivitätsdiagramm

UML Aktivitätsdiagramme erlauben, **Abläufe** (d.h. Prozesse, Geschäftsprozesse, technische Prozesse, Nutzerverhalten, Vorgänge …) darzustellen mit:

- **Steuerflüssen** (Sequenz, if-Verzweigung, merge),
- **Nebenläufigkeit** (fork, join).
- **Datenflüssen** (Eingabe-, Ausgabe-Objekte),
- **Organisationseinheiten** (swim lanes),



https://www.uml-diagrams.org/activity-diagrams.html

Use Cases vs. Aktivitätsdiagramme

Für Use Cases verwendet man UML Aktivitätsdiagramme zur:

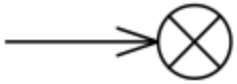
- grafischen Darstellung des Ablaufs eines Szenarios und zur
- Ergänzung zu textuellen Use Cases und Use Case Diagrammen.

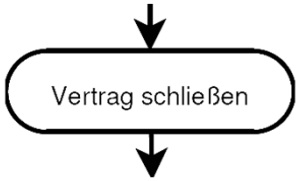
Weiterhin können dargestellt werden:

- **Geschäftsprozesse**, z.B. Schritte zur Eröffnung eines Kontos bei einer Bank.
- **technische Prozesse**, z.B. der Ablauf in einem Programm (jeder Algorithmus und jedes Programm kann als UML Aktivitätsdiagramm dargestellt werden).

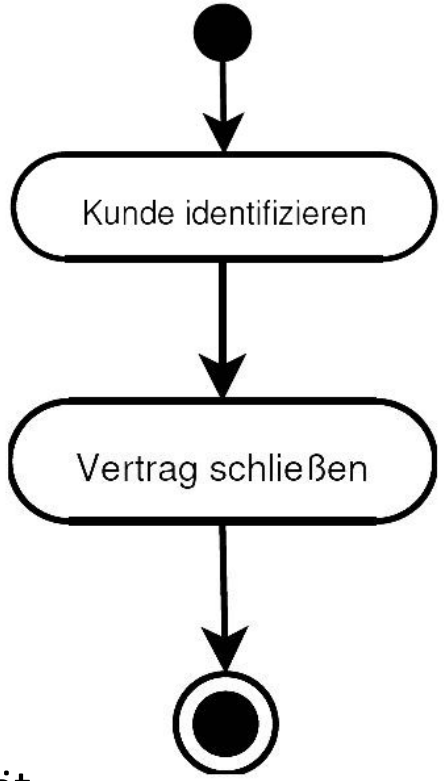
Grundelemente, Sequenz

Sequenz ist eine Folge von Aktivitäten mit:

- **Start**: 
- **Ende:**  regulär  Abbruch.
- Rechteck-Ovale symbolisieren **Aktivitäten**.
- **Wörter** in Ovalen **benennen Aktivitäten**, nicht Objekte oder Zustände.

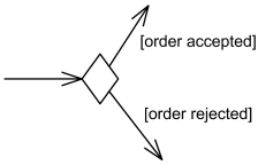


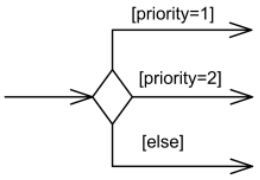
- **Pfeile** symbolisieren mögliche Flussrichtungen der Aktivität.



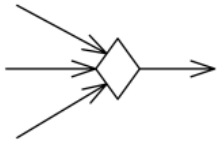
Verzweigung

- Rauten sind **Verzweigungspunkte** für Entscheidungen (decisions):
  - **ausgehende Pfeile** mit **Bedingungen** markieren.
  - Aktivität folgt Pfeil, dessen Bedingung erfüllt ist.
  - Bedingungen müssen sich ausschließen.

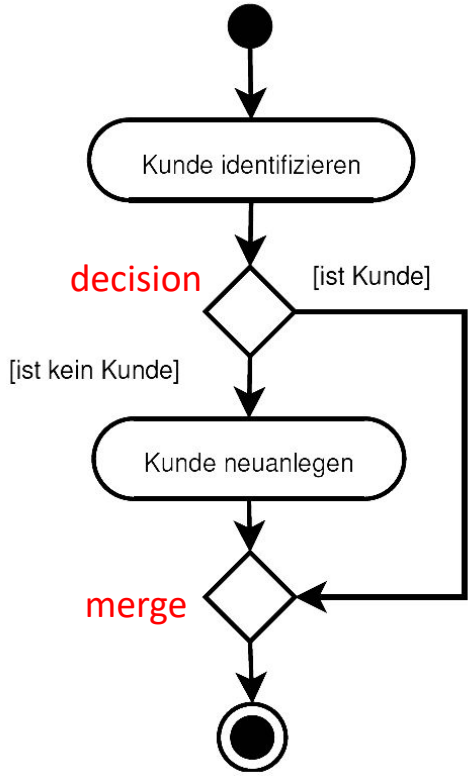




- **„Merge“**-Rauten sind Zusammenführungen alternativer Pfade (die gern vergessen werden).
  - Nach einem Merge wird fortgesetzt, wenn eine eingehende Aktivität das Merge erreicht hat (ODER).



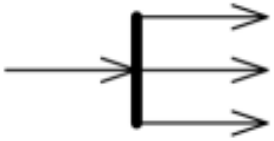
- Hinweis: In eine Aktivität sollte nur jeweils ein Pfeil hinein und einer wieder herausführen. Decision / Merge verwenden, um das zu erreichen.



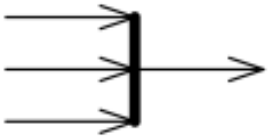
Nebenläufigkeit (Concurrency)

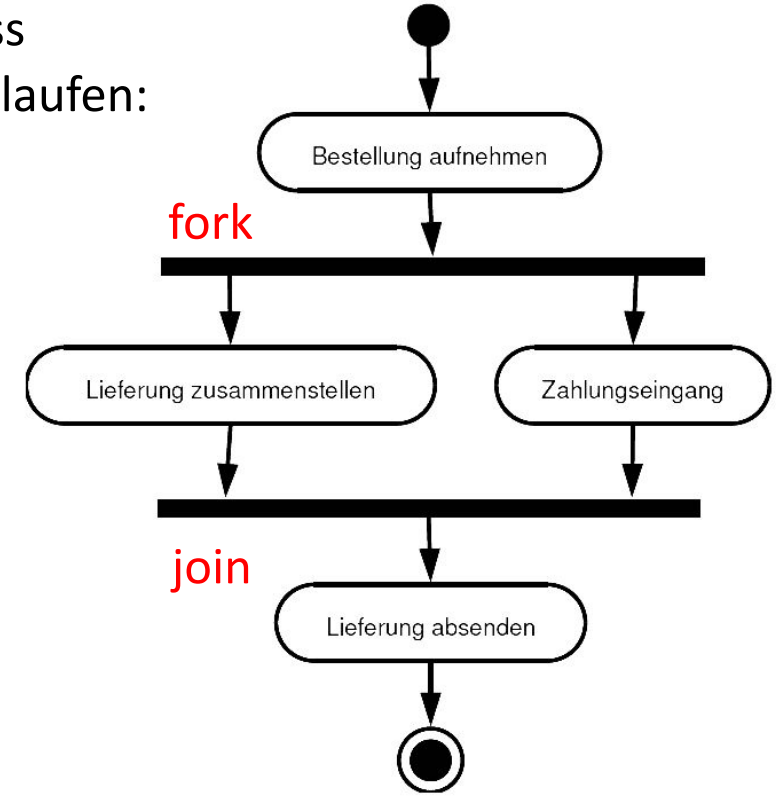
Nebenläufigkeit (engl. *concurrency*) bedeutet, dass mehrere Aktivitäten unabhängig “nebeneinander“ ablaufen:

- in beliebiger Reihenfolge oder
- gleichzeitig (parallel).



- **Fork** startet mehrere nebenläufige Aktivitäten, die alle unabhängig fortsetzen.
- **Join** führt die vorher mit fork aufgespalteten Aktivitäten wieder zusammen.
  Nach einem join wird fortgesetzt, wenn alle eingehenden Aktivitäten das join erreicht haben.
  Man nennt dies auch UND-join oder UND-Synchronisation.

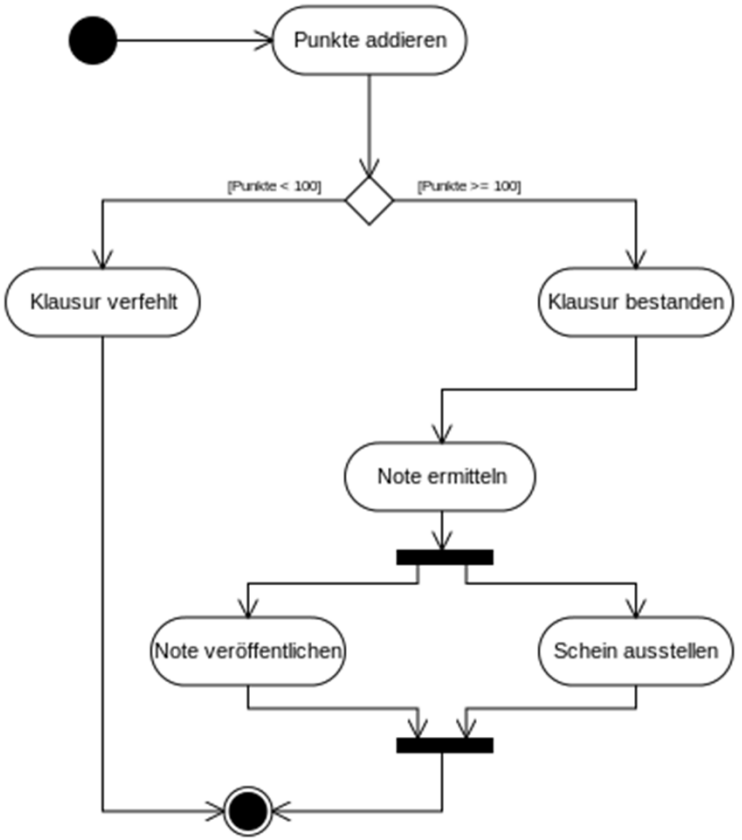




Verzweigung - Beispiel

Beschreiben Sie in Stichpunkten den Ablauf.

- **1.** Die Auswertung einer Klausur beginnt mit dem Addieren der Punkte.
- **2.a** Wenn weniger als 100 Punkte erreicht wurden, wird die Aktion „Klausur verfehlt“ ausgeführt. Dann ist die Aktivität beendet.
- **2.b** Sind >= 100 Punkte erreicht, wird die Aktion „Klausur bestanden“ ausgeführt.
- **3.** Anschließend wird die Aktion „Note ermitteln“ ausgeführt.
- **4.** Wenn die Note ermittelt ist, beginnen zeitgleich zwei Aktivitäten „Note veröffentlichen“ und „Schein ausstellen“.
- **5.** Sind beide Aktivitäten aus 4. beendet, ist die Gesamt-aktivität beendet.

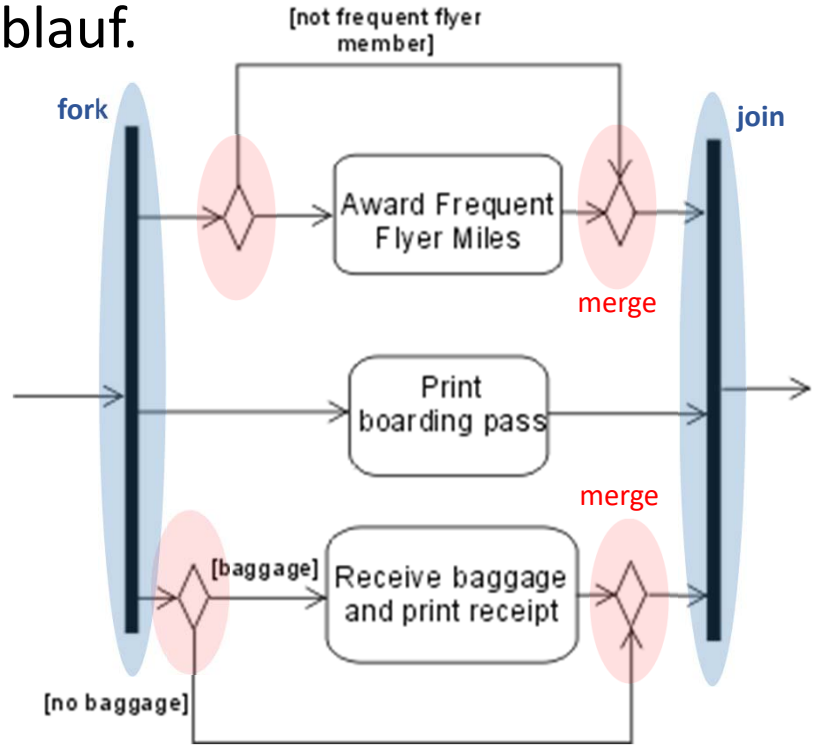


Nebenläufigkeit - Beispiel

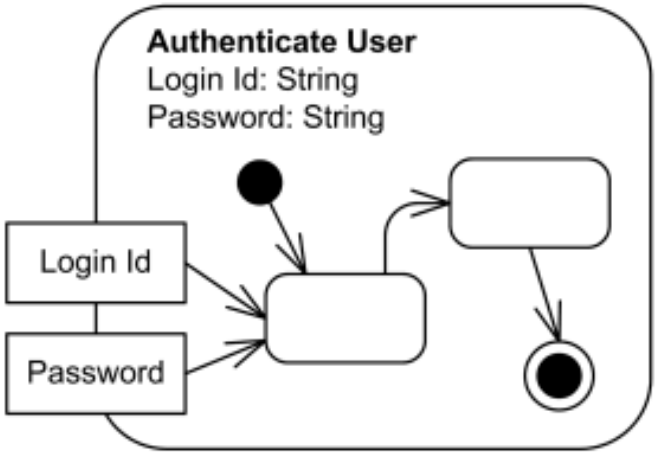
Flughafen Check-in. Beschreiben Sie den Ablauf.

- **1.** Beim Check-In starten am Schalter zeitgleich drei Aktivitäten (fork):
- **2.a** Wenn der Fluggast Meilen sammelt, werden die Flugmeilen gewährt.
- **2.b** Die Bordkarte wird gedruckt.
- **2.c** Wenn der Fluggast Gepäck einchecken möchte, wird der Gepäckabschnitt ausgedruckt.
- **3.** Wenn alle drei Aktivitäten abgeschlossen sind, ist der Check-in Prozess beendet.

Bei 2.a und 2.c ist zu beachten, dass die Verzweigungen vor dem join wieder gemerged werden müssen.

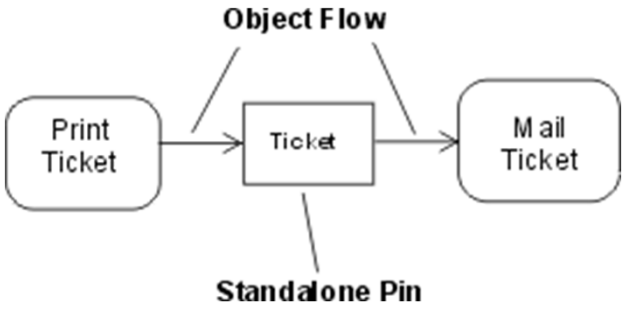


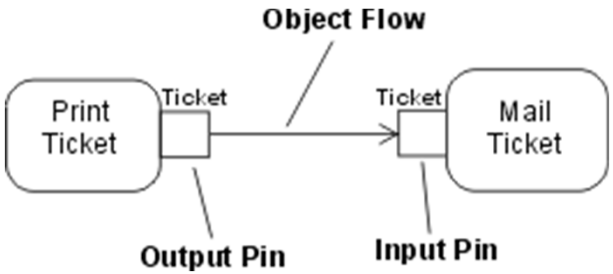
Object Flow



- **Objekte (Daten):** als Darstellung entweder als Rechteck zwischen Aktivitäten oder als “pins” an Aktivitäten.

- **Aktionen (Schritte):** gerundete Rechtecke.

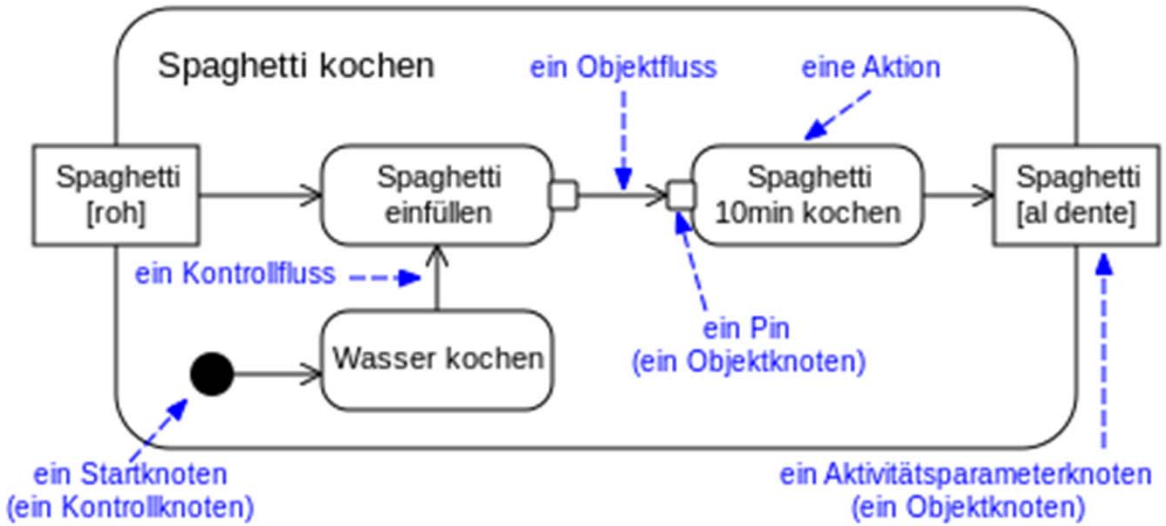




Beispiel: Object Flow

Das Beispiel zeigt:

- Objekte
  - Spaghetti [roh]
  - Spaghetti [al dente]
- Aktionen
- Start-Knoten
- Abgrenzung als umschließende Aktion: “Spaghetti kochen”.

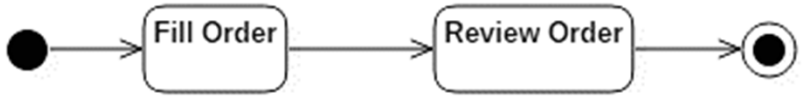


Prozess vs. Workflow

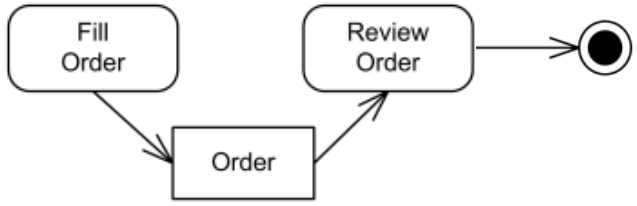
- **Prozess:** Folge von Aktionen: A1, A2, A3 …

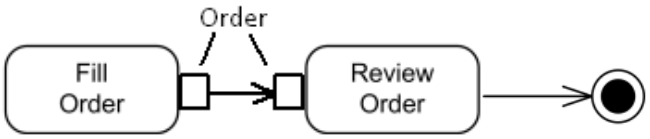
  Es “fließt” der Zustand, in dem sich der Prozess befindet:

  A1 → A2 → A3



- **Workflow:** Arbeitsgegenstände (Objekte, Pins) fließen durch eine Folge von Stationen (vgl. Fließband).



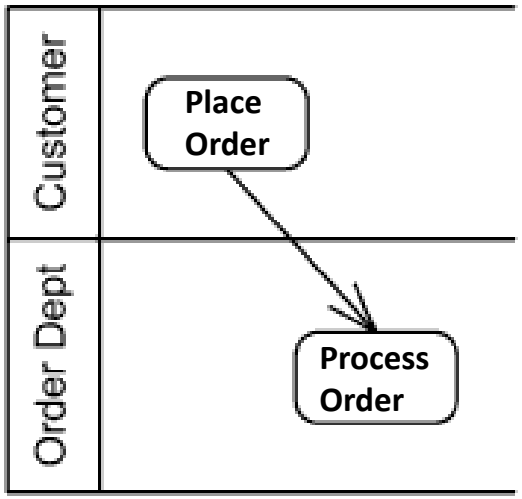


Swim Lanes

Swim Lanes dienen der organisatorischen Zuordnung von Aktivitäten.

Sie repräsentieren Organisationen, Verantwortlichkeiten oder technische Komponenten, die für jeweilige Aktivitäten verantwortlich sind.

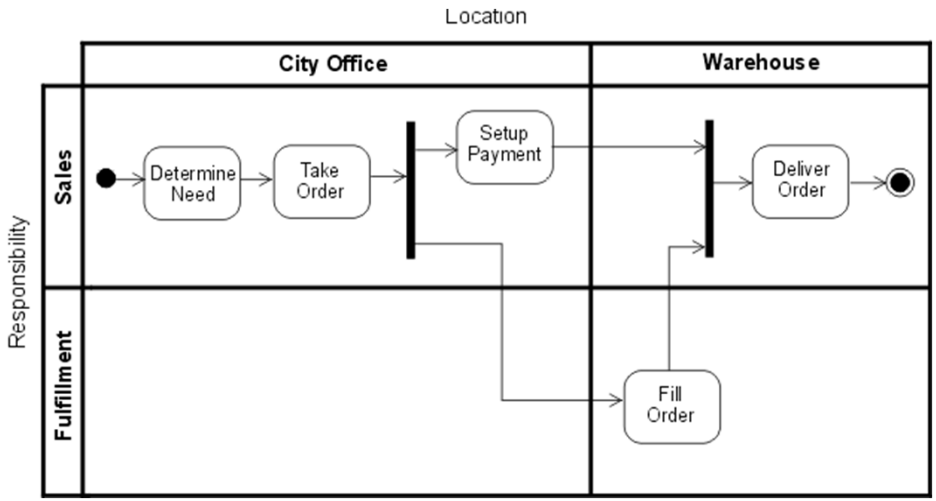
- Swim Lanes führen zu einer besseren Übersicht in Aktivitätsdiagrammen. Zudem können Verantwortlichkeiten zugeordnet werden.
- Swim Lanes werden als rechteckige Bereiche dargestellt.
- Im Beispiel führt die Customer-Organisation eine Aktivität “Place Order“ aus, welche im Order Department von einer Aktivität “Process Order“ verarbeitet wird.



Swim Lanes - Beispiel

Zwei-dimensionale Swim Lanes können z.B. Verantwortlichkeiten und Lokationen zeigen.

- **1.** Im City Office der Sales Abteilung wird ein Bedarf ermittelt.
- **2.** Anschließend wird dort eine Bestellung angenommen.
- **3.** Im Anschluss werden zeitgleich (nebenläufig) zwei Aktivitäten 4.a und 4.b angestoßen (fork).
- **4.a** Das City Office von Sales veranlasst den Bezahlvorgang.
- **4.b.** Gleichzeitig wird im Lager der Fulfillment-Abteilung die Lieferung zusammengestellt.
- **5.** Wenn die Bezahlung erfolgt und die Lieferung zusammengestellt ist (join), wird die Bestellung im Lager ausgeliefert.



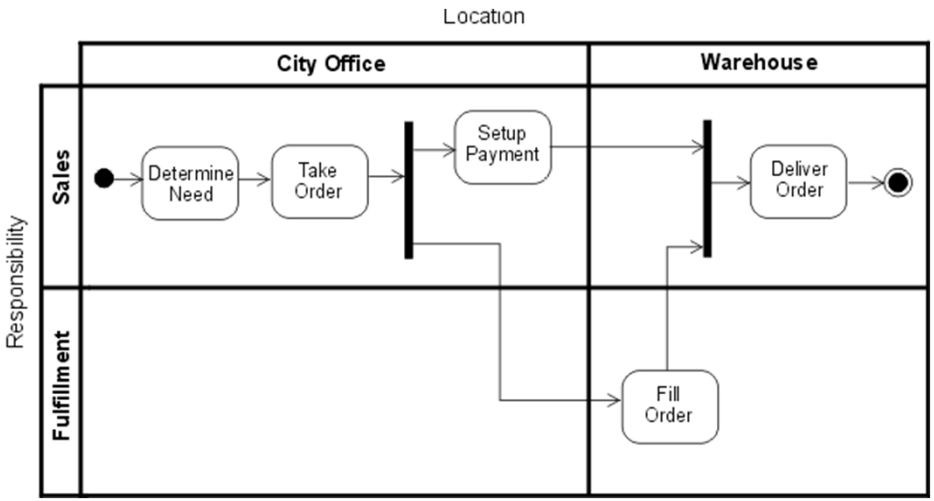
Was geschieht bei Fehlern?

Aktivitätsdiagramme zeigen in der Regel den „Normalfall“, den „Happy Path“.

Problematisch wird es, wenn Fehler auftreten.

Welche Probleme können im Beispiel auftreten?

- Die Bezahlung erfolgt nicht.
  - Wer oder wie wird das bemerkt?
  - Was soll in diesem Fall geschehen?
  - Was geschieht mit der ggf. schon zusammengestellten Lieferung im Lager?
- Im Lager kann die Lieferung nicht komplett zusammengestellt werden. Es fehlen Teile.
  - Was soll passieren, wenn der Kunde schon bezahlt hat?
  - Was passiert mit der unfertigen Lieferung im Lager?
  - Soll diese wieder ausgepackt werden?
  - Kann es zu „race conditions“ kommen? -- https://de.wikipedia.org/wiki/Race_Condition.



Unterbrechungen

Aktivitäten, die auf Eingaben warten oder von sich aus „lange“ dauern, können ein System blockieren (z.B. wenn ein Nutzer nicht reagiert).

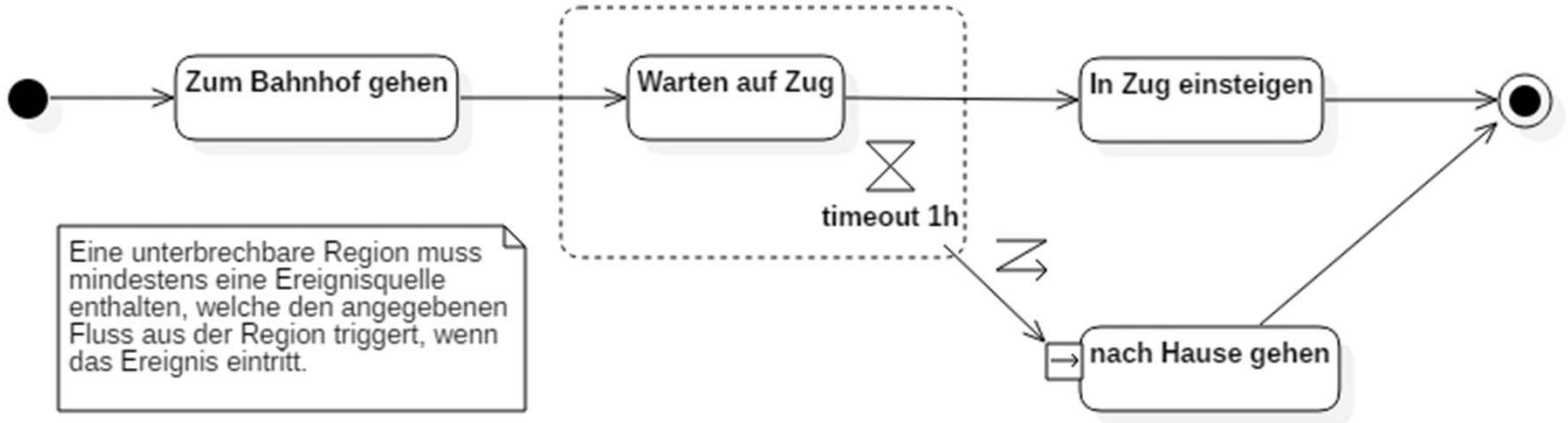
- UML bietet für diese Fälle sogenannte „unterbrechbare Regionen“.
- Unterbrechungen werden durch Signale ausgelöst, die in einer unterbrechbaren Region als Quellen eingezeichnet werden.
- Signale können jederzeit (asynchron) auftreten, z.B. das Time Event Signal.
- Tritt ein Signal ein, wird nicht mit der regulären Folge-Aktivität aus der blockierten Aktivität fortgesetzt, sondern mit der dem Signal zugeordneten Folge-Aktivität, die sich außerhalb der unterbrechbaren Region befinden muss.
- Alle Aktivitäten innerhalb der Region werden daraufhin abgebrochen und die Region mit dem vom Signal ausgehenden Fluss verlassen.

Unterbrechungen, Beispiel

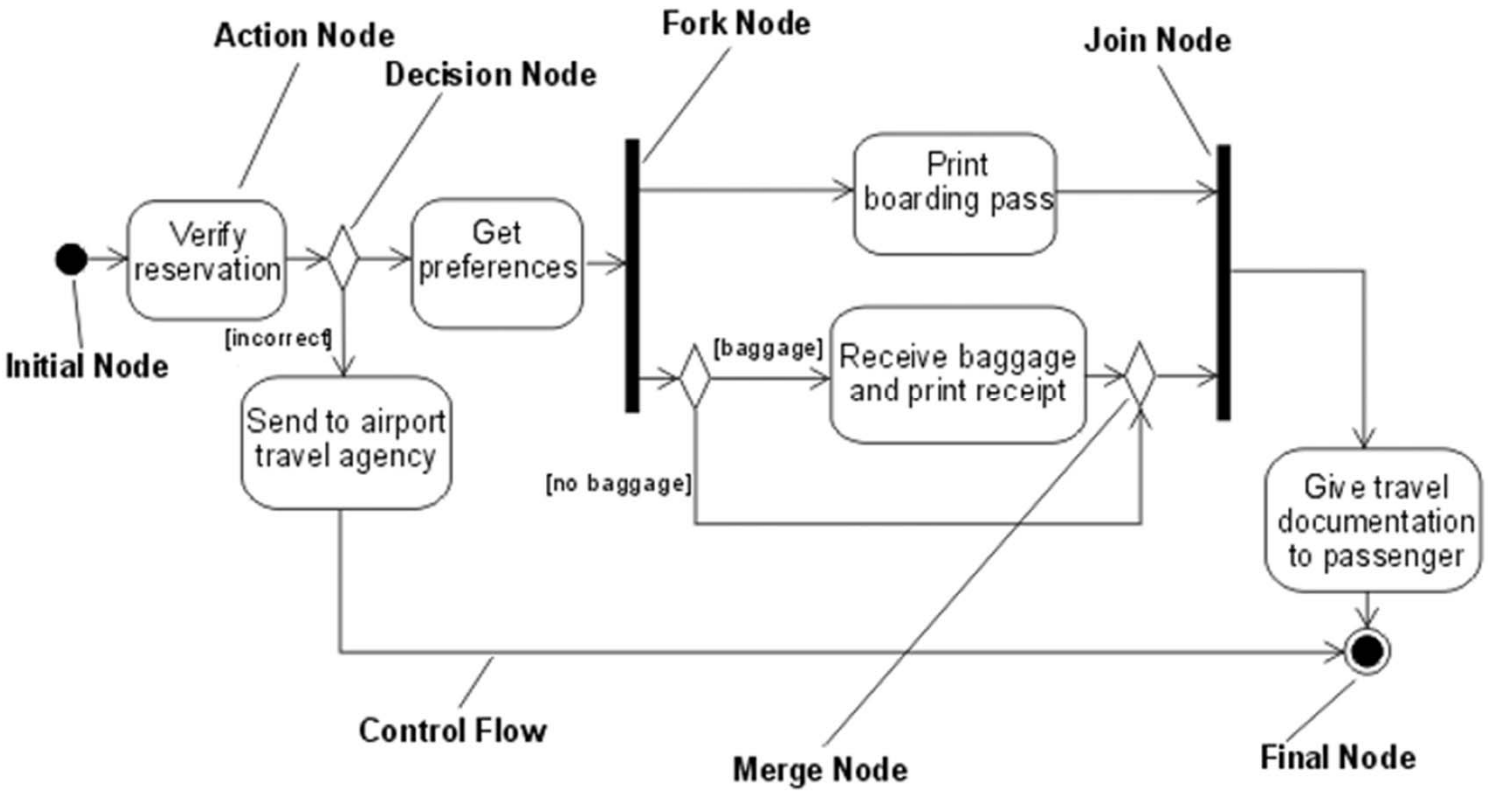
Das Beispiel zeigt eine Aktivität „Warten auf Zug“, die ggf. endlos andauern kann.

Die Aktivität wird deshalb in eine „unterbrechbare Region“ gezeichnet.

Entweder kommt der Zug oder das Time Event Signal (Symbol der Sanduhr) tritt nach 1h auf. Dann wird das Warten abgebrochen und die unterbrechbare Region mit dem vom Time Event Signal ausgehenden Fluss, d.h. mit „nach Hause gehen“ fortgesetzt.



Beispiel: Fluggastabfertigung



Zusammenfassung

